

Two ways past the wall

Extending clifft beyond its 2^k limit

A · Stabilizer-rank backend

change the *representation*:

pay for magic, not for dimensions

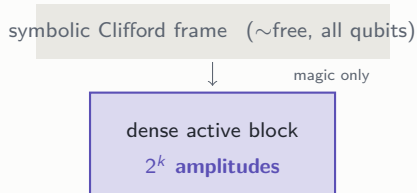
B · GPU shot batching

change the *hardware use*:

run thousands of shots abreast

clifft in thirty seconds

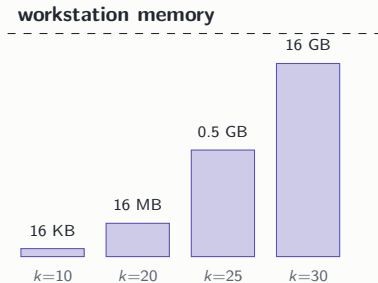
- Clifford gates are “easy”: clifft tracks them **symbolically**, almost for free, however many qubits.
- Only genuine *magic* (T gates) forces it to hold real amplitudes: a dense array of 2^k complex numbers, the **active block**.
- k counts how much magic is live at once — usually far smaller than the qubit count. That is why clifft is fast.



One engine, two tiers: symbolic where physics allows it, dense only where it must be.

The wall: 2^k doubles with every step

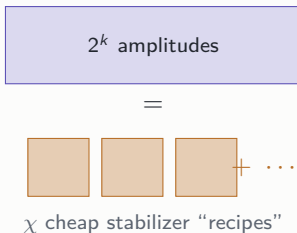
- Memory doubles per unit of k :
 $k = 30 \Rightarrow 16$ GB, $k = 40 \Rightarrow 16$ TB.
- clifft's compiler already fights hard for small k (measurement-driven reduction)
— circuits that still land at large k are *genuinely* dense.
- So the remaining question is not
“compile better” but: **what do we do**
when 2^k itself is the cost?



Two independent bets on what to do about it — one mathematical, one architectural.

A · The idea: pay for magic, not for dimensions

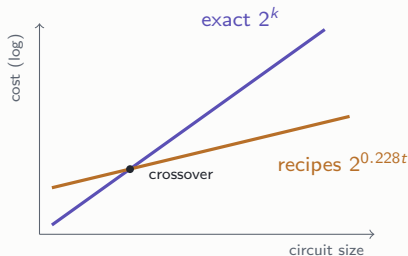
- States with *little* magic are secretly simple: they are a **short sum of “free” stabilizer states** — the states Clifford simulators handle natively.
- Store χ small recipes (a few KB each) instead of 2^k raw numbers. Like describing an image as a few overlapping patterns instead of listing every pixel.
- Clifford gates update all recipes cheaply; only T gates grow their number; **measurements shrink it back.**



The cost driver becomes the amount of magic t , not the dimension 2^k .

A · Why it should work

- There is an optimal way to split a T gate so the number of recipes grows only as $2^{0.228t}$ — far slower than 2^k when magic is sparse.
- Accepting a small, **tunable** error δ lets you keep only a random sample of recipes ($\sim 1/\delta^2$ of them). Measured accuracy is *predictable*: error = 0.50δ , on the nose.
- Everything stays in the language cliff already speaks: same frame idea, same measurement collapse.



Trade exactness you can measure for reach you cannot otherwise have.

A · Evidence (measured, against cliff's real fast path)

- Every component validated to **machine precision**; results checked against an exact independent method we built for the purpose.
- Head-to-head on hard dense circuits: the backend overtakes exact cliff at $n \approx 34$ (coarse accuracy) and $n \approx 48$ (8% error).
- Past $n \approx 62$ it is the **only method that runs at all**.

$n = 72$	memory	time
any exact method	~ 1 TB	—
rank backend	1.9 GB	267 s

Accuracy dial, measured:
error = 0.50δ across the whole sweep
(halve the error $\Rightarrow 4\times$ the work).

The niche is real — and honestly bounded: below the crossover, exact cliff stays the right tool.

A · Where it wins, where it doesn't

Small circuits	Large, magic-sparse	Adaptive, Clifford-heavy
cliff exact is faster <i>and</i> exact — keep it.	The niche. Only feasible method past the memory wall (approximate, dial-controlled).	Frame pays off once Clifford depth $\gtrsim$ block width ($8\times$ and growing past the boundary); the standard open-source tool cannot run these circuits at all.

- Also measured where it *loses*: at low Clifford depth the frame's bookkeeping costs $\sim 2\times$ — the advantage is a measured phase boundary, not a slogan.

Status: validated prototype (Python + C++), honest benchmark note; next — one more external baseline and a natural adaptive workload, then a methods paper.

B · The idea: don't run one shot faster — run 4,096 abreast

- A cliff gate just *streams* a small array (16 KB–16 MB). One such array cannot keep a GPU busy: **a stadium choir asked to sing one sentence.**
- But cliff's workloads are Monte-Carlo: **thousands of shots of the same program**, today run one after another.
- Put $B=4,096$ shots' arrays on the GPU side by side and apply every gate to *all of them* in one launch.

CPU today: single file



GPU: all at once



one instruction, B statevectors

Batching doesn't make the GPU faster — it makes the problem big enough for the GPU to be fast.

B · Why it should work (1): shots march in lockstep

- The shape of a shot — which operation runs when, and how big the array is at every step — is **fixed at compile time and identical for every shot**. clifft's compiler already writes this schedule down.
- What differs per shot (measurement outcomes, noise) is just **a few numbers**, which enter the shared computation as per-shot parameters — like SIMD lanes, one lane per shot.
- So the classic objection to batching — “shots diverge” — *does not apply to clifft*. Its structure is unusually batchable.
- And nobody serves this niche: the two existing batched-GPU simulators both **fall back to one-shot-at-a-time** exactly when circuits measure mid-flight — clifft's home turf.

The hardest architectural question resolves in the design's favor before any GPU code is written.

B · Why it should work (2): the speed limit is bandwidth — and GPUs have $7\times$ more

- These operations do almost no arithmetic per byte: on *any* machine they run at the speed of memory.
 - Measured: clifft on 11 CPU cores is only $2.6\times$ one core — **the bus is the limit, not the cores.**
 - A GH200's memory is $\sim 7\times$ wider than a server CPU's. Once the batch saturates it: **$5\text{--}10\times$ more shots per second**, honestly bounded.
-

memory bandwidth (TB/s)



server CPU ~ 0.5

GH200 GPU ~ 3.5

big published “ $100\times$ ” numbers use weak baselines — we keep a refuted-claims list and quote the physics.

The claim is deliberately modest — which is why we can trust it.

B · The one risk, and the \$20 experiment that settles it

- Mid-circuit measurements need a decision from the CPU: results cross to the host, outcomes cross back. Each crossing costs microseconds — **the same scale as the small kernels themselves**.
- Two reasons it should survive: the crossing is paid **once per batch** (nanoseconds per shot at $B=4,096$), and the GH200's CPU-GPU link is built precisely to make it cheap.
- A microbenchmark faithful to cliff's kernels — *including complete measurements with the round-trip* — is built, validated, and waiting. GH200 time: \$2/hour, self-serve.

round-trip measured cheap **go**: batched backend; first target = replay-style queries (no round-trips at all)

round-trip eats the win **no-go**, cleanly — or on-device sampling is the follow-up

Either answer is cheap, quantitative, and final — that is what the microbenchmark is for.

How the two bets fit together

A · Stabilizer rank extends *reach*

circuits cliff cannot hold at all today —
approximate, with a measured accuracy dial

- Both work *because of* cliff's structure, not despite it: the symbolic frame, the small active block, and the compile-time shot schedule are exactly what decompositions and batching need.
- Both were reviewed adversarially; every headline number above survived a fair baseline.

Next: A — external baseline (Quokka#) + a natural adaptive workload, then a methods paper. B — the GH200 run, then batched replay queries.

B · GPU batching extends *throughput*

the circuits cliff already runs —
many more shots per second, exact as ever